

Chap 4.3 딥러닝 알고리즘

Deep Learning Algorithms

[컴퓨터비전특론]

일시: 2026. 04. 14.

발표자: 이진영 (AM202616703)

4.3 딥러닝 알고리즘

이진영 | AM202616703

딥러닝 알고리즘은 심층 신경망을 사용한다는 공통점이 있습니다.

머신 러닝 알고리즘처럼 목적에 따라 합성곱 신경망(CNN), 순환 신경망(RNN), 제한된 볼츠만 머신(RBM), 심층 신뢰 신경망(DBN)으로 분류됩니다.

4.3.1

심층 신경망 (DNN)

4.3.2

합성곱 신경망 (CNN)

4.3.3

순환 신경망 (RNN)

4.3.4

제한된 볼츠만 머신
(RBM)

4.3.5

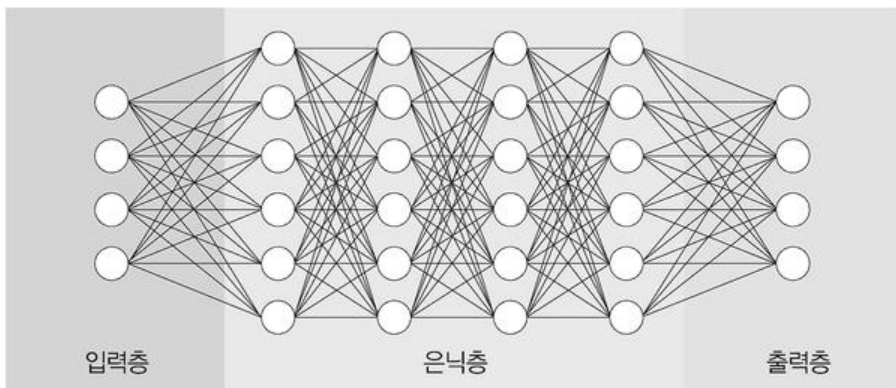
심층 신뢰 신경망
(DBN)

4.3.1 심층 신경망 (DNN)

이진영 | AM202616703

심층 신경망 (DNN)

- 입력층과 출력층 사이 다수의 은닉층을 포함하는 인공 신경망
- 다수의 은닉층을 추가했기 때문에 별도의 트릭 없이 비선형 분류가 가능
- 다양한 비선형적 관계를 학습할 수 있는 장점이 있음
- 단점: 학습을 위한 연산량이 많고, 기울기 소멸 문제가 발생할 수 있음
 - 드롭아웃, 렐루 함수, 배치 정규화 등을 적용하여 해결



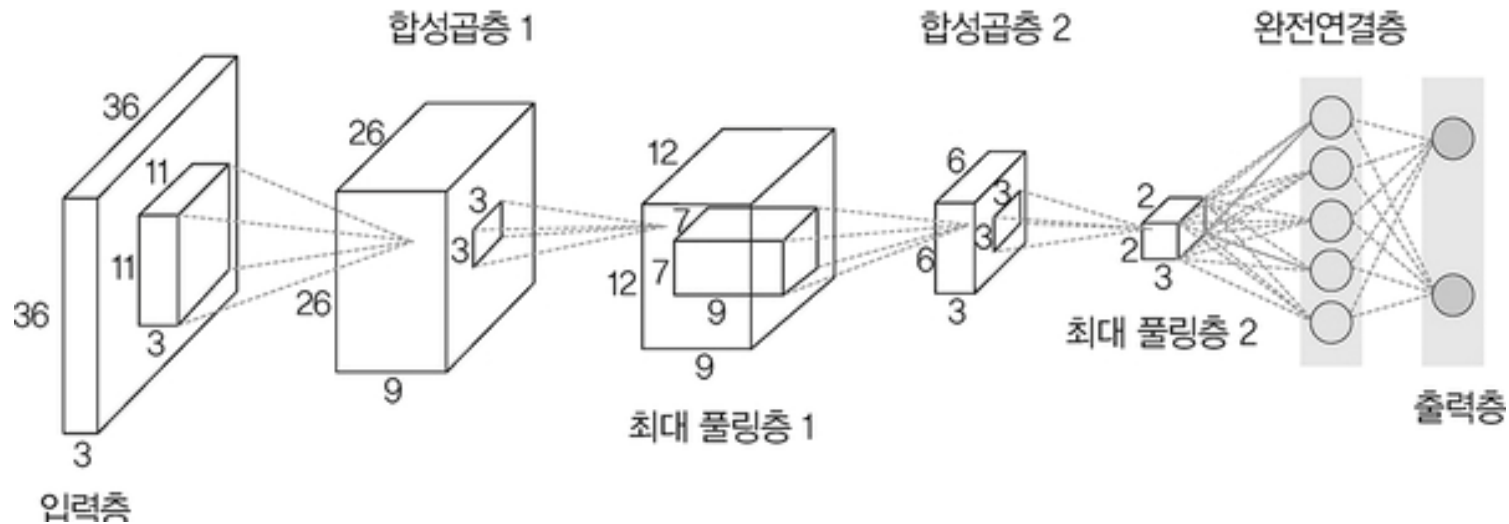
4.3.2 합성곱 신경망 (CNN)

이진영 | AM202616703

합성곱 신경망 (CNN)

- 합성곱층, 풀링층을 포함하는 이미지 처리 성능이 좋은 인공 신경망 알고리즘
- 영상 및 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체 위치를 찾아내는 데 유용
- 이미지에서 객체, 얼굴, 장면을 인식하기 위해 패턴을 찾는 데 특히 유용

대표 모델: LeNet-5 · AlexNet · VGG · GoogLeNet · ResNet



4.3.2 합성곱 신경망 (CNN)

이진영 | AM202616703

합성곱 신경망 (CNN)

- 합성곱층, 풀링층을 포함하는 이미지 처리 성능이 좋은 인공 신경망 알고리즘
- 영상 및 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체 위치를 찾아내는 데 유용
- 이미지에서 객체, 얼굴, 장면을 인식하기 위해 패턴을 찾는 데 특히 유용

대표 모델: LeNet-5 · AlexNet · VGG · GoogLeNet · ResNet

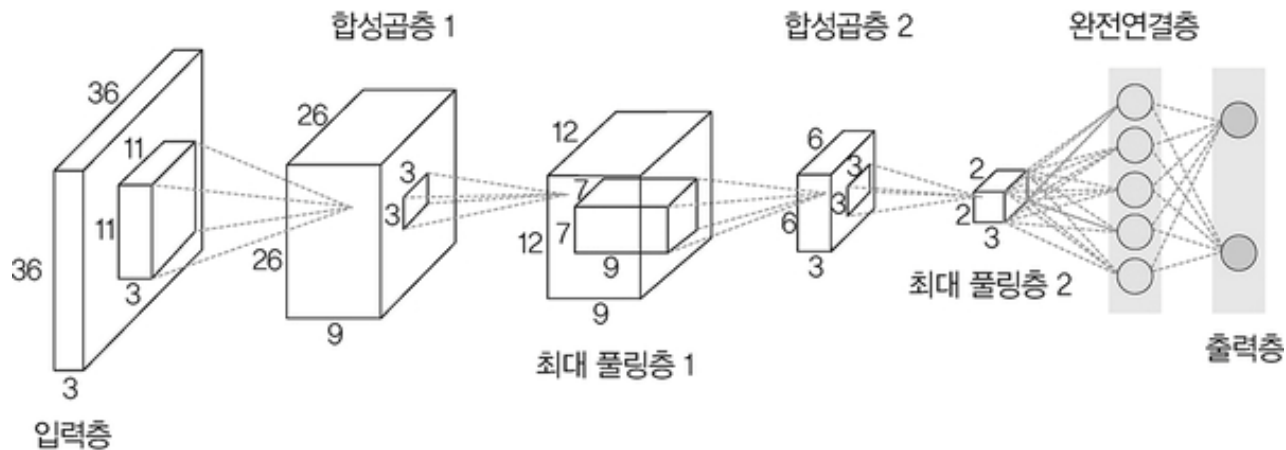
기존 신경망과의 차별성

- 각 층의 입출력 형상을 유지
- 이미지의 공간 정보를 유지하면서 인접 이미지와 차이가 있는 특징을 효과적으로 인식
- 복수 필터로 이미지의 특징을 추출하고 학습
- 추출한 이미지의 특징을 모으고 강화하는 풀링층이 있음
- 필터를 공유 파라미터로 사용하기 때문에 일반 신경망 대비 학습 파라미터가 매우 적음

4.3.2 합성곱 신경망 (CNN)

이진영 | AM202616703

CNN 아키텍처



주요 특징

- 이미지 공간 정보 유지
- 복수 필터로 특징 추출
- 풀링층으로 특징 강화
- 공유 파라미터로 효율적 학습
- 비선형 분류 가능

4.3.3 순환 신경망 (RNN)

이진영 | AM202616703

순환 신경망 (RNN)

- 시계열 데이터 (음악, 영상 등) 같은 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망
- 순환: 자기 자신을 참조, 현재 결과가 이전 결과와 연관이 있다는 의미

순환 신경망의 특징

- 시간성을 가진 데이터가 많음
- 시간성 정보를 이용하여 데이터의 특징을 잘 다룸
- 시간에 따라 내용이 변하므로 데이터는 동적이고, 길이가 가변적
- 매우 긴 데이터를 처리하는 연구가 활발히 진행
- 기울기 소멸 문제 → 메모리 개념을 도입한 LSTM으로 보완

활용 분야

- 언어 모델링

- 텍스트 생성

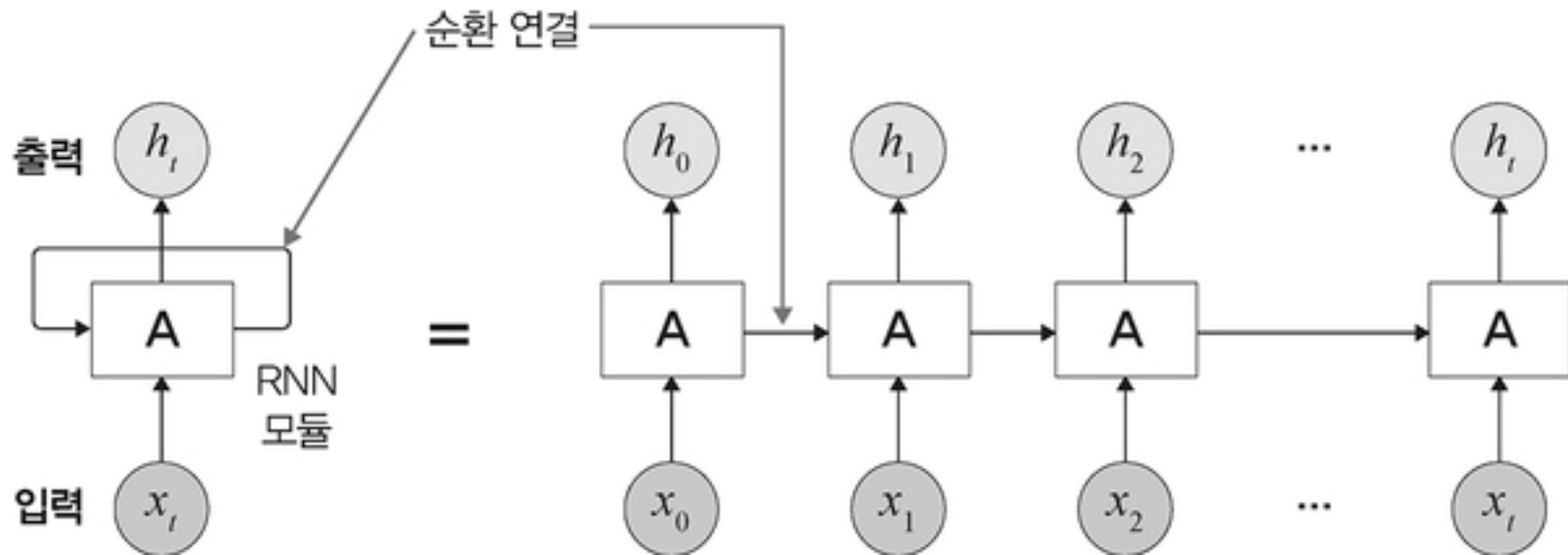
- 자동 번역

- 음성 인식

- 이미지 캡션

4.3.3 순환 신경망 (RNN)

이진영 | AM202616703



4.3.4 제한된 볼츠만 머신 (RBM)

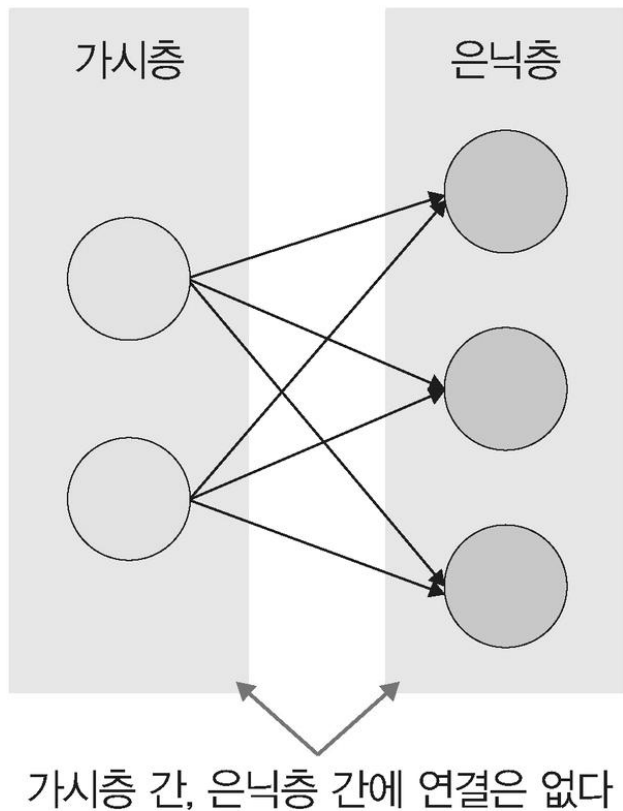
이진영 | AM202616703

제한된 볼츠만 머신

- 가시층(visible layer), 은닉층(hidden layer)으로 구성된 모델
- 가시층은 은닉층과만 연결됨
(가시층 $\leftrightarrow$ 가시층, 은닉층 $\leftrightarrow$ 은닉층 연결 없음)
 - 이것이 제한된 볼츠만 머신 (Restricted Boltzmann Machine)

제한된 볼츠만 머신의 특징

- 차원 감소, 분류, 선형 회귀 분석, 협업 필터링, 특성 값 학습, 토픽 모델링에 사용
- 기울기 소멸 문제를 해결하기 위해 사전 학습 용도로 활용 가능
- 심층 신뢰 신경망(DBN)의 요소로 활용



4.3.5 심층 신뢰 신경망 (DBN)

이진영 | AM202616703

심층 신뢰 신경망 (Deep Belief Network, DBN)

- 입력층과 은닉층으로 구성된 제한된 볼츠만 머신을 블록처럼 여러 층으로 쌓은 형태
- 사전 훈련된 RBM을 층층이 쌓아 올린 구조로, 레이블이 없는 데이터에 대한 비지도 학습이 가능
- 부분적인 이미지에서 전체를 연상하는 일반화와 추상화 과정을 구현할 때 유용

학습 절차

- 1 가시층과 은닉층 1에 제한된 볼츠만 머신을 사전 훈련
- 2 첫 번째 층 입력 데이터와 파라미터를 고정하여 두 번째 층 RBM을 사전 훈련
- 3 원하는 층 개수만큼 RBM을 쌓아 올려 전체 DBN을 완성

DBN 구조



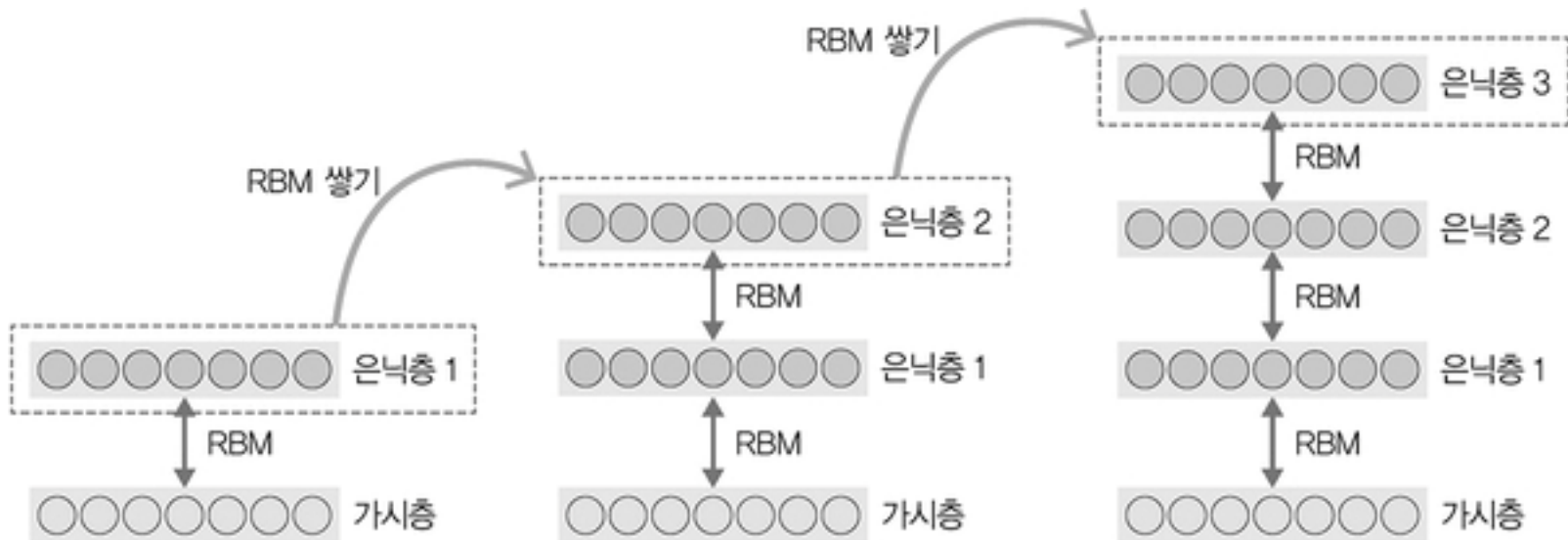
↑ 위로 올라갈수록 추상적 특성 추출

4.3.5 심층 신뢰 신경망 (DBN)

이진영 | AM202616703

심층 신뢰 신경망의 특징

- 순차적으로 학습시켜 가면서 계층적 구조를 생성 / 비지도 학습으로 학습
- 위로 올라갈수록 추상적 특성 추출 / 학습된 가중치를 다층 퍼셉트론의 초기값으로 활용



4.3 딥러닝 알고리즘 - 요약

이진영 | AM202616703

DNN 심층 신경망	다수의 은닉층 비선형 분류 가능 드롭아웃·ReLU·배치정규화 필요
CNN 합성곱 신경망	이미지 처리 특화 합성곱층 + 풀링층 LeNet, AlexNet, VGG, GoogLeNet, ResNet
RNN 순환 신경망	시계열 데이터 처리 자기 참조 구조 LSTM으로 기울기 소멸 문제 보완
RBM 제한된 볼츠만 머신	가시층 + 은닉층 사전 학습 용도 DBN의 구성 요소
DBN 심층 신뢰 신경망	RBM 적층 구조 비지도 학습 가능 부분→전체 추상화, 가중치 초기값 활용

공통점: 모두 심층 신경망(Deep Neural Network) 기반 알고리즘

4.4 우리는 무엇을 배워야 할까?

이진영 | AM202616703

머신 러닝 vs 딥러닝 선택 기준

- 주어진 데이터를 활용하여 **어떤 결과를 얻고 싶은지**에 따라 선택이 달라짐
- 간단한 선형 회귀·분류로 원하는 값을 도출할 수 있다면 머신 러닝만으로도 충분
- 복잡한 비선형 데이터의 분류 및 예측을 도출하고 싶다면 딥러닝으로 학습해야 함

머신 러닝

- ✓ 선형 회귀
- ✓ 분류
- ✓ 간단한 예측

vs

딥 러닝

- ✓ 비선형 분류
- ✓ 이미지 처리
- ✓ 복잡한 예측

핵심 관점

머신 러닝과 딥러닝은 우리가 얻고 싶은 결과를 도출하기 위한 도구(연장) 일 뿐입니다.

- 어떤 도구를 선택할 때 더 좋은 성능·효과를 얻을 수 있을지는 데이터를 수집하고 분석하는 사람만이 알 수 있습니다.

데이터를 활용하여 얻고자 하는 것에 따라 머신 러닝 또는 딥러닝을 선택해서 학습하고 훈련시키면 됩니다.

Chap 7.3 순환 신경망 (RNN)

[컴퓨터비전특론]

일시: 2026. 04. 14.

발표자: 이진영 (AM202616703)

7.3 순환 신경망 (RNN) — 개념

이진영 | AM202616703

RNN이란?

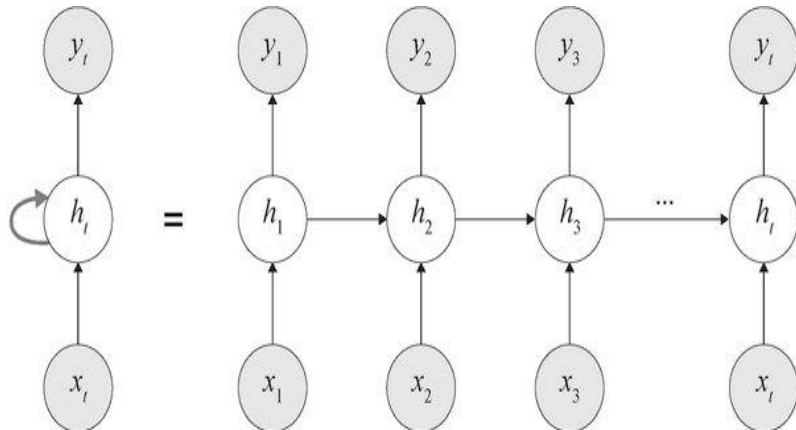
시간적으로 연속성이 있는 데이터를 처리하기 위해 고안된
인공신경망

- 기존 네트워크와 다르게 기억(memory)을 가짐
- 기억 = 현재까지 입력 데이터를 요약한 정보
- 새로운 입력 시마다 기억이 수정 → 최종 기억은 모든 입력을 요약

동작 원리

- 첫 입력(x_1) → 첫 기억(h_1) 생성
- 두 번째 입력(x_2) → 기존 기억(h_1) + 새 입력으로 h_2 생성
- 입력 길이만큼 반복 수행

RNN 구조



① RNN이란? — 기억이 있는 신경망

이진영 | AM202616703

일반 신경망 (기억 없음)

"재" → [신경망] → ???

"밧" → [신경망] → ???

"어" → [신경망] → ???

✗ 각 글자를 따로따로 봄
앞 글자를 기억하지 못함

RNN (기억 있음)

"재" → [RNN] → '재... 뭔가 시작'

"밧" → [RNN] → '재밧... 긍정적'

"어" → [RNN] → '재밧어! = 긍정'

기억
전달

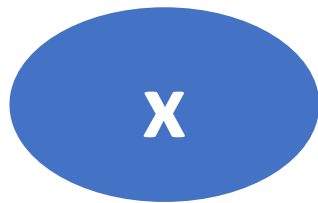
✓ 이전 글자를 기억하면서
다음 글자와 합쳐서 이해

RNN = 이전까지 읽은 내용을 기억하면서, 새 입력과 합쳐 다음을 이해하는 신경망

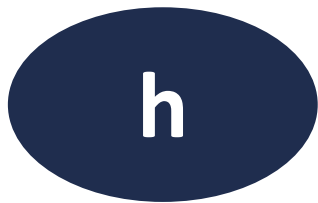
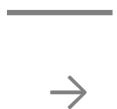
핵심 차이: 일반 신경망은 기억이 없고, RNN은 이전 정보를 기억하며 처리한다

② $x \cdot h \cdot y$ — 입력, 기억, 출력

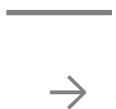
이진영 | AM202616703



입력 (Input)



기억 (Hidden State)



출력 (Output)

RNN에 들어오는 데이터
→ 문장이면 단어 하나하나
→ 음악이면 음표 하나하나

예) "이", "영화", "정말", "재밌다"

지금까지 읽은 내용을 요약한 것
→ 매 단계마다 업데이트됨
→ 이전 기억 + 새 입력 = 새 기억

예) '이 영화 정말...' 까지 읽은 상태

RNN이 최종적으로 내놓는 답
→ 다대일: 마지막에 딱 1번
→ 다대다: 매 단계마다

예) "긍정" / "부정" / "중립"

x (입력) $\rightarrow$ h (기억) $\rightarrow$ y (출력) | h 는 매 단계 갱신, y 는 최종 판단에 사용

x = 읽는 단어 | h = 지금까지 읽은 내용 요약 | y = 최종 판단 결과

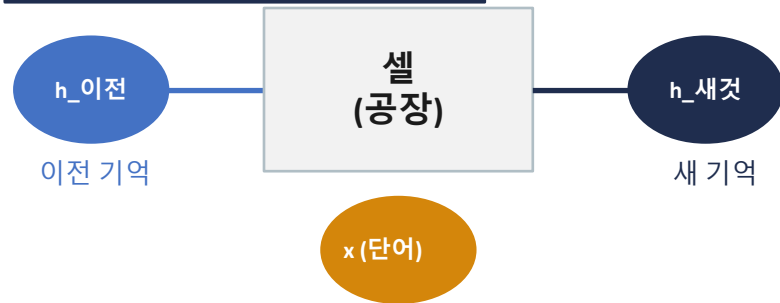
③ 셀(Cell) — 기억을 만드는 공장

이진영 | AM202616703

셀이 하는 일

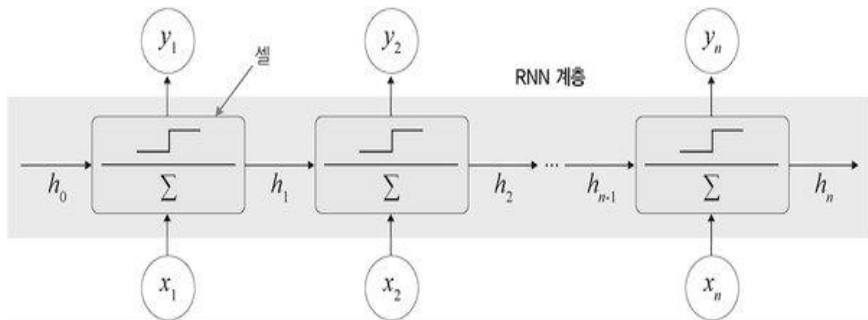
- 셀 = 한 번에 단어 하나(x)를 처리하는 단위
 - 이전 기억(h)과 새 입력(x)을 받아서
 - 새로운 기억(h)을 만들어 다음 셀에 넘김

셀 처리 과정

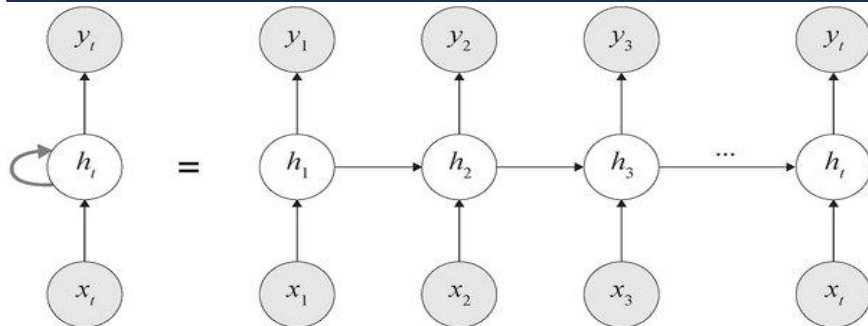


공장(셀)은 하나인데,
재료(x)와 이전 제품(h)을 넣으면 → 새 제품(h)이 나옴

실제 RNN 셀 구조



여러 셀이 이어진 모습



셀 = 이전 기억(h) + 현재 입력(x) → 새 기억(h)을 만드는 하나의 처리 단위

④ 전체 흐름 – "이 영화 정말 재밌다" 처리하기

이진영 | AM202616703



매 단계마다 기억(h)이 쌓이고, 마지막 h에 문장 전체 의미가 담겨 최종 판단(y)에 사용

⑤ 핵심 정리 – 한눈에 요약

이진영 | AM202616703

RNN이란?

이전 정보를
기억하면서 처리하는
신경망

x (입력)

RNN에 들어오는
데이터
(단어, 음표 등)

h (기억)

지금까지 읽은
내용을 요약한
메모리

y (출력)

최종 판단 결과
다대일: 마지막에
딱 1번 출력

셀(Cell)이란?

이전 기억(h) + 현재 입력(x) → 새 기억(h) 을 만드는 처리 단위
한 문장을 처리할 때 같은 셀을 반복해서 사용

한 줄 비유

책을 한 페이지씩 읽으며 내용을 기억하다가
마지막 페이지를 읽고 나서 '재밌다/ 별로다' 감상평을 쓰는 것

RNN = 기억(h)을 쌓아가며 순서 있는 데이터를 처리하는 신경망

7.3 RNN 입출력에 따른 형태

이진영 | AM202616703

유형	입력 / 출력	설명	대표 사례
일대일 one to one	$1 \rightarrow 1$	순환 없음 (일반 신경망)	순방향 신경망
일대다 one to many	$1 \rightarrow N$	입력 하나, 출력 다수	이미지 캡션
다대일 many to one	$N \rightarrow 1$	입력 다수, 출력 하나	감성 분석
다대다 many to many	$N \rightarrow N$	입력·출력 다수 (인코더-디코더)	자동 번역기
동기화 many to many	$N \rightarrow N$	입력과 출력이 동기화	비디오 프레임 분류

RNN은 입출력 구조에 따라 다양한 태스크(캡션·번역·감성분석·분류)에 활용 가능

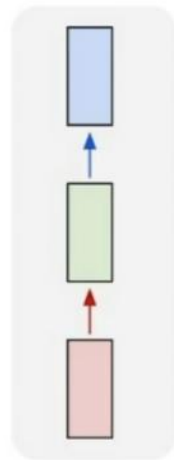
7.3 RNN 입출력에 따른 형태

이진영 | AM202616703

1. 일대일: 순환(loop) 구조가 없어 엄밀히는 RNN이 아님

- 입력 하나 → 출력 하나 (단일 입출력)
- 각 입력이 독립적으로 처리 → 이전 정보를 전혀 기억하지 않음
- 대표 사례: 순방향 신경망(Feedforward NN), 단일 이미지 분류

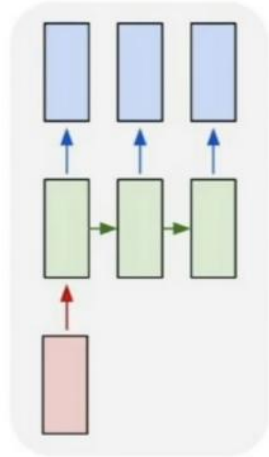
one to one



1. 일대다: 입력 하나 → 출력 다수 (1개 입력, N개 출력)

- 첫 timestep에만 입력을 받고, 이후 은닉 상태(h)만으로 출력을 연속 생성
- 대표 사례: 이미지 캡션 (Image Captioning)
- → 이미지 1장 입력 → "A dog is running" 같은 문장(다수 단어) 출력
- 추가 사례: 음악 생성 (첫 음표 1개 → 이후 멜로디 자동 생성).

one to many



7.3 RNN 입출력에 따른 형태

이진영 | AM202616703

3. **다대일:** 입력이 다수이고 출력이 하나인 구조로, 문장을 입력해서 이미지에 긍정/부정을 출력하는 감성 분석기에 사용됩니다.

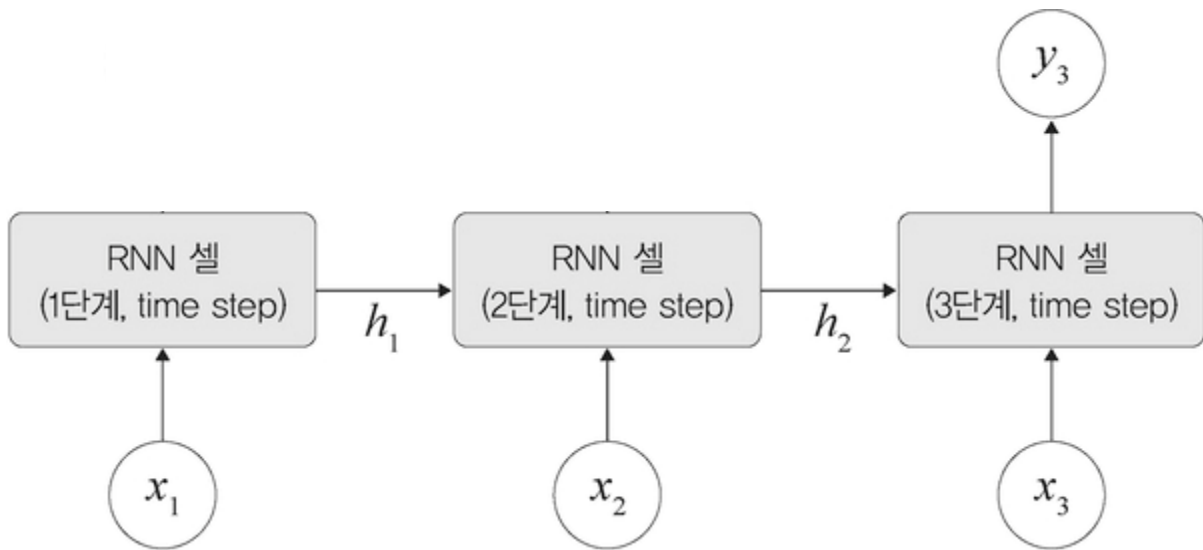
다대일에 대한 모델은 파이토치에서 다음과 같이 구현합니다. 다음은 예시 코드이며, 7장에서 다루는 모든 예제가 다대일 구조를 사용하므로 전체 예제를 통해 사용 방법을 확인할 수 있습니다.

```
self.em = nn.Embedding(len(TEXT.vocab.stoi), embedding_dim) ----- 임베딩 처리  
self.rnn = RNNCell_Encoder(embedding_dim, hidden_size) ----- RNN 적용  
self.fc1 = nn.Linear(hidden_size, 256) ----- 완전연결층  
self.fc2 = nn.Linear(256, 3) ----- 출력층
```

7.3 RNN 입출력에 따른 형태

이진영 | AM202616703

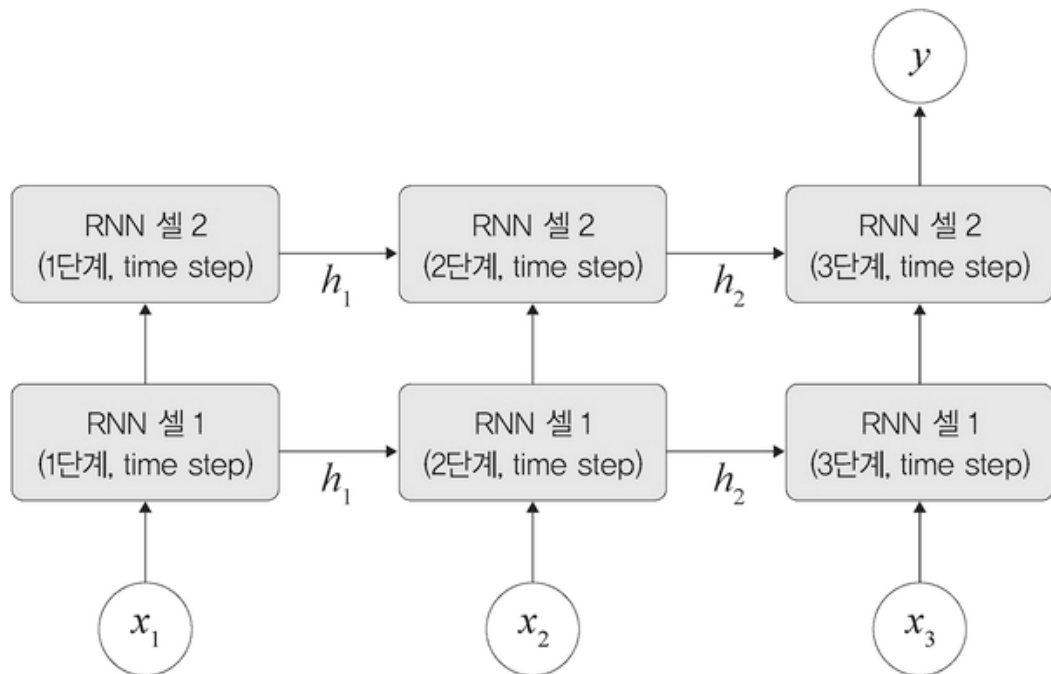
코드를 구조화하면 다음 그림과 같습니다. 하지만 코드는 입력과 출력 사이에 하나의 RNN 셀(Cell)만 가지고 있는 것에 주의해야 합니다.



7.3 RNN 입출력에 따른 형태

이진영 | AM202616703

또한, 다대일 구조에 층을 쌓아 올리면 다음과 같이 적층된 구조를 가질 수 있습니다.



RNN은 입출력 구조에 따라 다양한 태스크(캡션·번역·감성분석·분류)에 활용 가능

7.3 RNN 입출력에 따른 형태

이진영 | AM202616703

4. **다대다**: 입력과 출력이 다수인 구조로, 언어를 번역하는 자동 번역기 등이 대표적인 사례입니다. 예를 들어 다대다에 대한 모델은 파이토치에서 **시퀀스-투 시퀀스**로 구현합니다. 그런데 파이토치는 텐서플로처럼 구현이 간단하지 않습니다.

텐서플로에서는 `keras.layers.SimpleRNN(100, return_sequences=True, name='RNN')`처럼 `return_sequences=True` 옵션으로 시퀀스를 리턴하도록 쉽게 구현할 수 있습니다. 반면에 파이토치에서는 문장 번역에서 많이 사용되는 **시퀀스-투-시퀀스(seq2seq)**를 이용하는 방식으로 사용됩니다.

Seq2Seq 완전 이해

입력 문장을 읽어서 → 다른 문장으로 출력하는 구조 (예: 번역기)

①

Seq2Seq란?

개념과 등장 배경

②

인코더

문장을 읽고 요약

③

디코더

요약본으로 출력 생성

④

전체 흐름

인코더+디코더 합체

⑤

파이토치 코드

코드로 이해하기

개념 → 구조 → 파이토치 코드까지 한번에

① Seq2Seq란? — 문장을 문장으로

이진영 | AM202616703

Sequence-to-Sequence : 순서 있는 데이터(문장)를 입력받아 → 다른 순서 있는 데이터(문장)를 출력

번역기 예시로 이해하기



활용 분야



자동 번역

한국어 → 영어



챗봇

질문 → 답변



문서 요약

긴 글 → 짧은 요약



음성 인식

음성 → 텍스트

Seq2Seq = 문장 → 문장 변환 구조 | 번역·챗봇·요약 등 다양한 분야에 활용

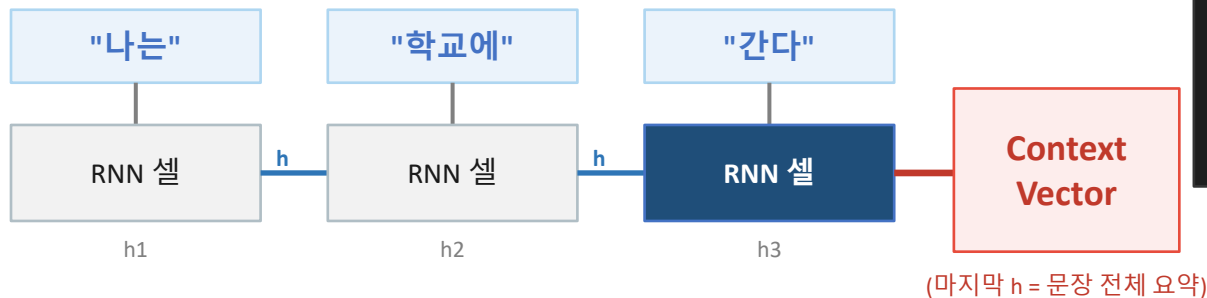
② 인코더 (Encoder) — 문장을 읽고 요약

이진영 | AM202616703

인코더의 역할

- 입력 문장을 단어 하나씩 RNN으로 읽음
 - 매 단계마다 기억(hidden state)을 업데이트
 - **마지막 hidden state = Context Vector (문장 전체 요약본)**
 - 이 Context Vector 하나만 디코더에 넘겨줌

인코더 처리 흐름



인코더 코드

```
class Encoder(nn.Module):
    def __init__(self,
                  input_size, # 단어 수
                  embed_size, # 임베딩 크기
                  hidden_size): # 기억 크기
        self.embed = nn.Embedding(
            input_size, embed_size)
        self.rnn = nn.GRU(
            embed_size, hidden_size)

    def forward(self, x):
        embedded = self.embed(x)
        _, hidden = self.rnn(embedded)
        return hidden # Context Vector
```

📖 비유: 책 전체를 다 읽고 나서 "이 책은 학교 가는 이야기야" 라고 한 문장으로 요약하는 것

인코더 = 입력 문장 전체를 읽고 Context Vector(요약본) 하나를 만들어 디코더에 전달

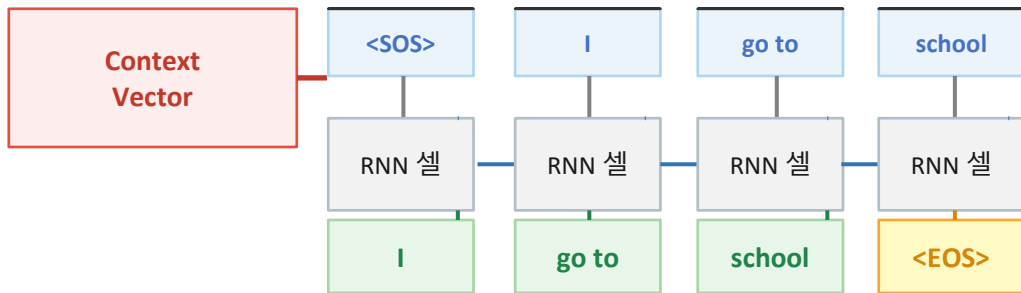
③ 디코더 (Decoder) — 요약본으로 출력 생성

이진영 | AM202616703

디코더의 역할

- 인코더에서 받은 Context Vector를 초기 hidden state로 사용
 - <SOS> 토큰(시작 신호)부터 출발해서 단어를 하나씩 생성
 - 이전에 출력한 단어를 다음 입력으로 사용 (자기 참조)
 - <EOS> 토큰(종료 신호)이 나오면 멈춤

디코더 단어 생성 흐름



← 이전 출력이 다음 입력이 됨

🚦 <SOS> = 번역 시작 신호 | <EOS> = 번역 종료 신호

디코더 코드

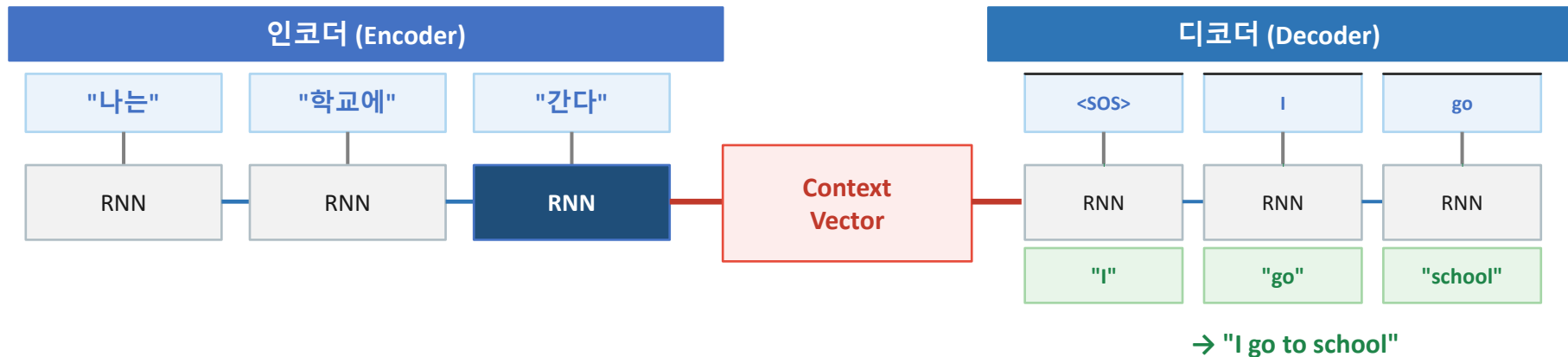
```
class Decoder(nn.Module):
    def __init__(self,
                  output_size,
                  embed_size, hidden_size):
        self.embed = nn.Embedding(
            output_size, embed_size)
        self.rnn = nn.GRU(
            embed_size, hidden_size)
        self.fc = nn.Linear(
            hidden_size, output_size)

    def forward(self, x, hidden):
        emb = self.embed(x.unsqueeze(1))
        out, hidden = self.rnn(emb, hidden)
        pred = self.fc(out.squeeze(1))
        return pred, hidden
```

디코더 = Context Vector를 받아 <SOS>부터 시작해 단어를 하나씩 출력, <EOS>에서 종료

④ 전체 흐름 – 인코더 + 디코더 합체

이진영 | AM202616703



Seq2Seq 동작 3단계

1 인코더가 읽기

입력 문장을 RNN으로 단어씩 읽어
마지막 hidden = Context Vector 생성

2 Context 전달

인코더의 마지막 hidden state를
디코더의 초기 hidden으로 전달

3 디코더가 출력

<SOS>부터 시작해 단어를 하나씩
생성, <EOS>가 나오면 종료

인코더가 요약 → Context Vector 전달 → 디코더가 출력 생성 = Seq2Seq 전체 흐름

⑤ 파이토치 코드 — Seq2Seq 합치기

이진영 | AM202616703

Seq2Seq 클래스 코드

```
class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder

    def forward(self, src, trg):
        # 1. 인코더: 입력 문장 읽기
        hidden = self.encoder(src)

        # 2. 디코더: 한 단어씩 출력
        input = trg[:, 0] # <SOS> 토큰
        for t in range(1, trg_len):
            output, hidden = \
                self.decoder(input, hidden)
            outputs[:, t] = output
            input = output.argmax(1)
        return outputs
```

코드 핵심 포인트

① 인코더 → hidden

인코더가 문장 전체를 읽고
hidden(Context Vector)을 반환

② <SOS>로 시작

디코더의 첫 입력은 항상
<SOS> (번역 시작 신호)

③ 이전 출력 → 다음 입력

output.argmax(1)로 예측한
단어를 다음 input으로 사용

모델 생성 예시

```
enc = Encoder(1000, 256, 512)
dec = Decoder(1000, 256, 512)
model = Seq2Seq(enc, dec)
```

Seq2Seq = Encoder + Decoder | 인코더가 Context 생성 → 디코더가 단어씩 출력

Seq2Seq 전체 요약

이진영 | AM202616703

Seq2Seq	문장 → 문장 변환 구조 (many-to-many)	번역, 챗봇, 요약, 음성인식
인코더	입력 문장을 RNN으로 읽어 Context Vector 생성	마지막 hidden = 요약본
Context Vector	인코더의 마지막 hidden state = 문장 전체 의미 압축	디코더의 초기 hidden으로 전달
디코더	<SOS>부터 시작, 이전 출력을 다음 입력으로 단어 생성	<EOS>가 나오면 종료
<SOS> / <EOS>	<SOS> = 번역 시작 신호 / <EOS> = 번역 종료 신호	디코더 제어 토큰
코드 핵심	hidden=encoder(src) → for문으로 decoder 반복 호출	output.argmax(1) → 다음 input

인코더가 문장을 요약 → Context Vector 전달 → 디코더가 단어씩 출력 → Seq2Seq 완성

7.3 RNN 입출력에 따른 형태

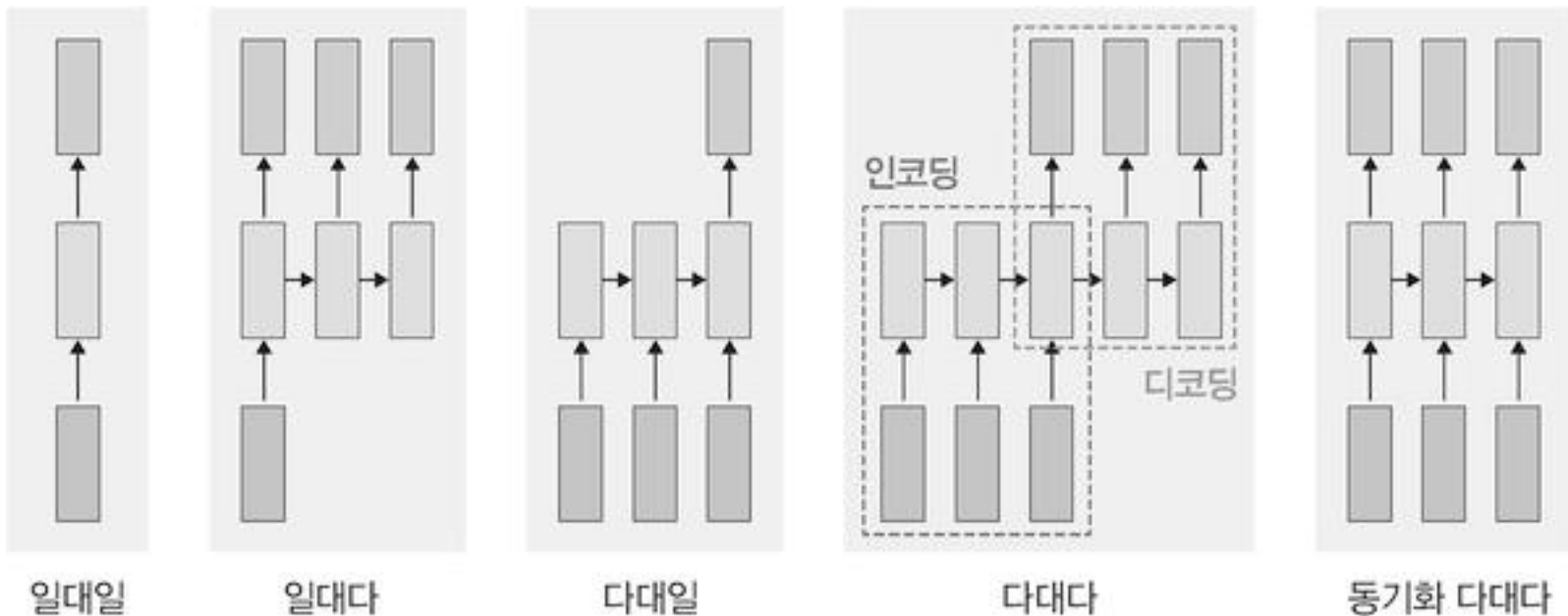
이진영 | AM202616703

5. 동기화 다대다: 4(다대다)의 유형처럼 입력과 출력이 다수인 구조입니다. 문장에서 다음에 나올 단어를 예측하는 언어 모델, 즉 프레임 수준의 비디오 분류가 대표적 사례입니다.

7.3 RNN 입출력에 따른 형태

이진영 | AM202616703

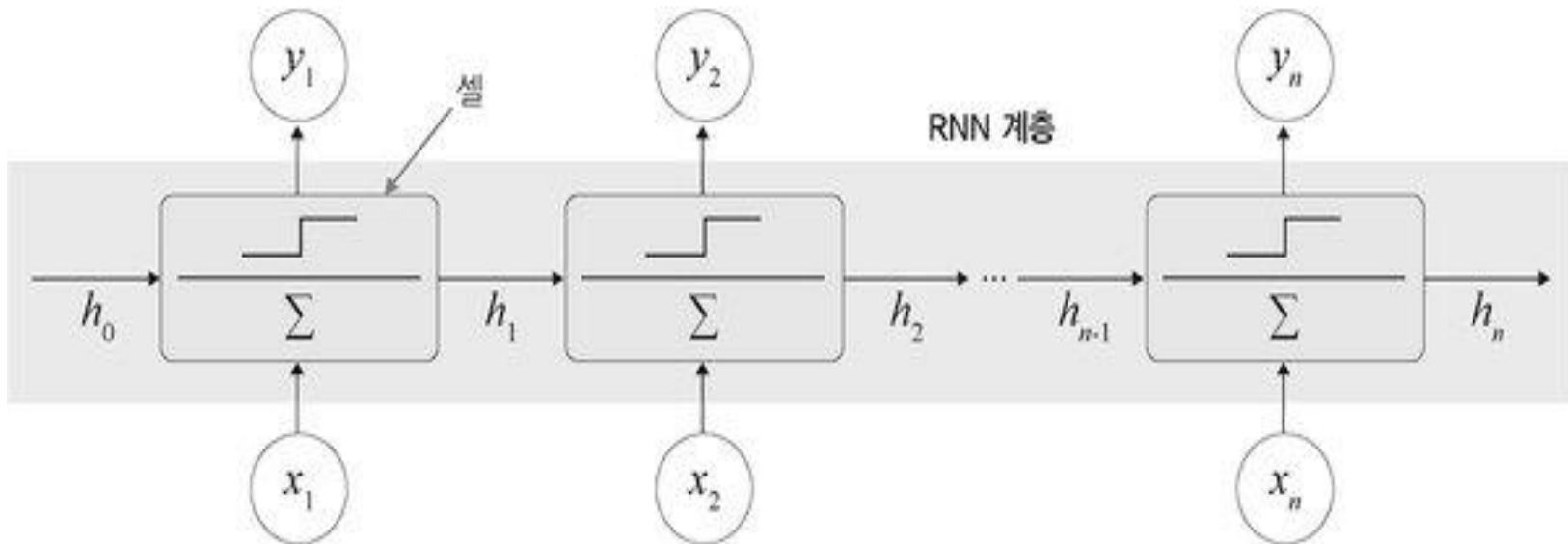
다음 그림은 앞서 언급된 순환 신경망 구조들을 그림으로 표현한 것입니다.



7.3.1 RNN 계층과 셀

이진영 | AM202616703

RNN은 내장된(built-in) 계층뿐만 아니라 셀 레벨의 API도 제공합니다. RNN 계층이 입력된 배치 순서대로 모두 처리하는 것과 다르게 RNN 셀은 오직 하나의 단계(time step)만 처리합니다. 따라서 순서대로 모두 처리하는 것과 다르게 RNN 셀은 오직 하나의 단계(time step)만 처리합니다. 따라서 RNN 셀은 RNN 계층의 for loop 구문을 갖는 구조라고 할 수 있습니다.



7.3.1 RNN 계층과 셀

이진영 | AM202616703

RNN 계층은 셀을 래핑^①하여 동일한 셀을 여러 단계에 적용합니다. 즉 셀은 실제 계산에 사용되는 RNN 계층의 구성 요소로, 단일 입력과 과거 상태(state)를 가져와서 출력과 새로운 상태를 생성합니다.

참고로 셀 유형은 다음과 같습니다.

- `nn.RNNCell`: SimpleRNN 계층에 대응되는 RNN 셀
- `nn.GRUCell`: GRU 계층에 대응되는 GRU 셀
- `nn.LSTMCell`: LSTM 계층에 대응되는 LSTM 셀

래핑^①: `dnn`이나 API를 사용하기 쉽도록 한 번 더 `dnn` 등으로 만들어 주는 것

7.3.1 RNN 계층과 셀

이진영 | AM202616703

이렇게 RNN의 계층과 셀을 분리해서 설명하는 이유는 파이토치에서 이 둘을 분리해서 구현이 가능하기 때문입니다. 따라서 앞으로 진행될 RNN 예제는 이 둘을 분리해서 진행합니다.

RNN 활용 분야로는 대표적으로 '자연어 처리'를 꼽을 수 있습니다. 연속적인 단어들의 나열인 언어(자연어) 처리는 음성 인식, 단어의 의미 판단 및 대화 등에 대한 처리가 가능합니다. 이외에도 손글씨, 센서 데이터 등 시계열 데이터 처리에 활용됩니다.

Chap 7.4 RNN 구조

[컴퓨터비전특론]

일시: 2026. 04. 14.

발표자: 이진영 (AM202616703)

7.4 RNN 구조

이진영 | AM202616703

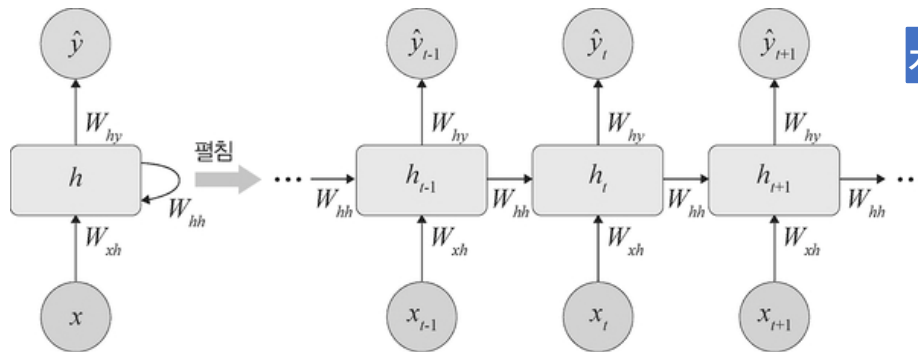
개요 & 가중치

RNN은 은닉층 노드들이 연결되어 이전 단계 정보를 은닉층 노드에 저장할 수 있게 구성한 신경망입니다.

x_{t-1} 에서 h_{t-1} 을 얻고, 다음 단계에서 h_{t-1} 과 x_t 를 사용하여 과거 정보와 현재 정보를 모두 반영합니다.

또한, h_t 와 x_{t+1} 의 정보를 이용하여 과거와 현재 정보를 반복해서 반영합니다.

이러한 구조를 요약한 것이 다음 그림의 오른쪽 부분과 같습니다.



가중치 3가지 (W_{xh} , W_{hh} , W_{hy})

- W_{xh} : 입력층 $\rightarrow$ 은닉층으로 전달되는 가중치
- W_{hh} : t 시점의 은닉층 $\rightarrow$ $t+1$ 시점의 은닉층으로 전달되는 가중치
- W_{hy} : 은닉층 $\rightarrow$ 출력층으로 전달되는 가중치

△ W_{xh} , W_{hh} , W_{hy} 는 모든 시점에서 동일합니다.
즉, 가중치를 공유하며 모든 가중치가 동일합니다.

RNN에서는 입력층, 은닉층, 출력층 외에 가중치를 세 개 가집니다. RNN의 가중치는 W_{xh} , W_{hh} , W_{hy} 로 분류됩니다.

RNN = 이전 단계 정보를 은닉층에 저장하여 과거와 현재 정보를 반복 반영하는 신경망

7.4 RNN 구조 — t 단계 계산 ① 은닉층 & ② 출력층

이진영 | AM202616703

t 단계 계산

① 은닉층 계산

은닉층 계산을 위해 x_t 와 h_{t-1} 이 필요합니다.

(이전 은닉층 × 은닉층 → 은닉층 가중치) + (입력층 → 은닉층 가중치 × 현재 입력 값) 으로 계산합니다.

수식: $h_t = \tanh(\hat{y}_t)$ / $\hat{y}_t = W_{hh} \times h_{t-1} + W_{xh} \times x_t$

$$h_t = \tanh(\hat{y}_t)$$

$$\hat{y}_t = W_{hh} \times h_{t-1} + W_{xh} \times x_t$$

② 출력층 계산

출력층은 심층 신경망과 계산 방법이 동일합니다.

(은닉층 → 출력층 가중치 × 현재 은닉층) 에 소프트맥스 함수를 적용합니다.

수식: $\hat{y}_t = \text{softmax}(W_{hy} \times h_t)$

$$\hat{y}_t = \text{softmax}(W_{hy} \times h_t)$$

7.4 RNN 구조 — t 단계 계산 ③ 오차

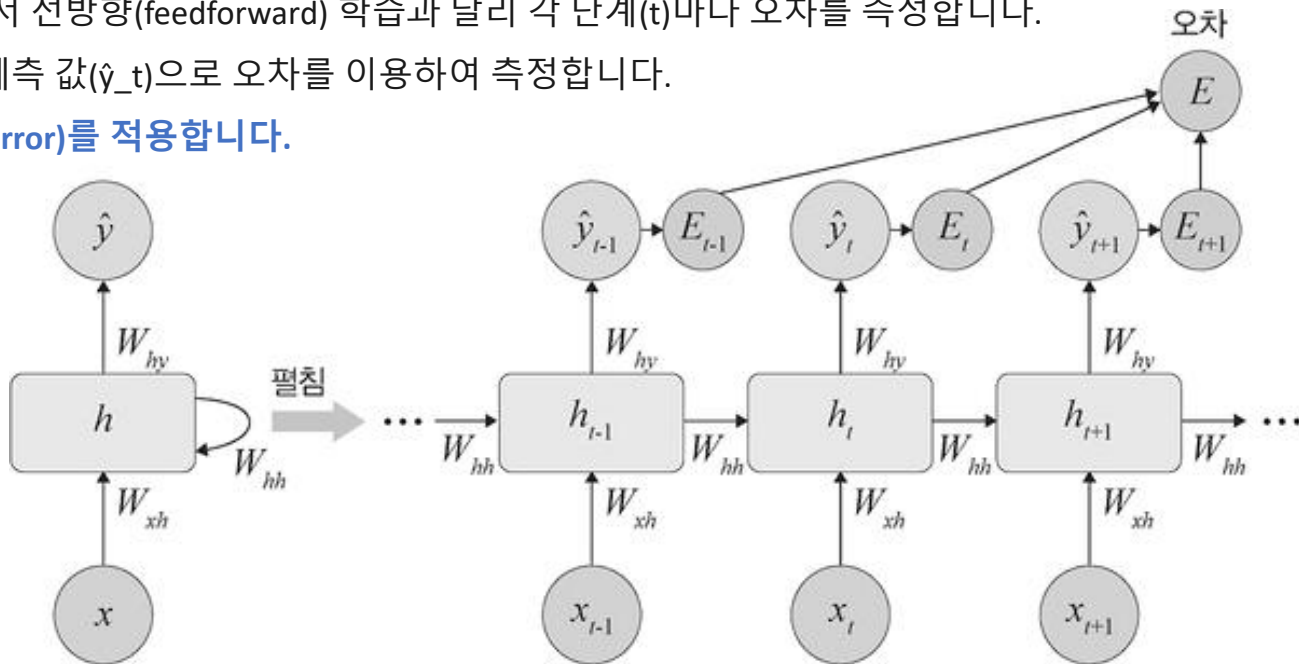
이진영 | AM202616703

t 단계 계산

③ 오차(E) 계산

RNN의 오차(E)는 심층 신경망에서 전방향(feedforward) 학습과 달리 각 단계(t)마다 오차를 측정합니다.
즉, 각 단계마다 실제 값(y_t)과 예측 값($\hat{y}_t$)으로 오차를 이용하여 측정합니다.

→ 평균 제곱 오차(mean square error)를 적용합니다.



7.4 RNN 구조 — t 단계 계산 ④ 역전파(BPTT)

이진영 | AM202616703

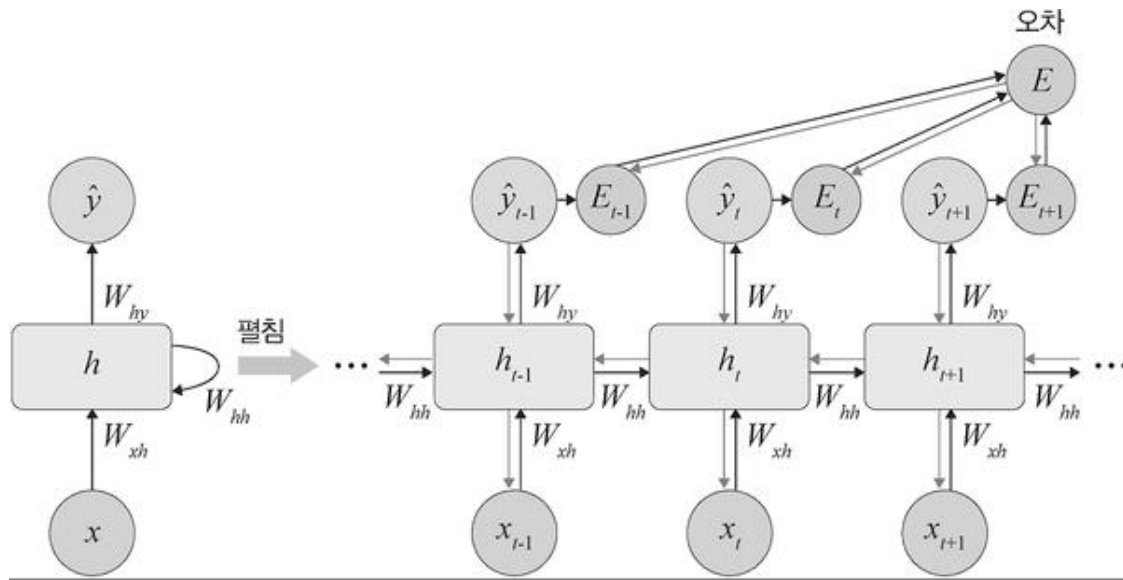
t 단계 계산

④ 역전파 계산 — BPTT

BPTT(BackPropagation Through Time)를 이용하여 모든 단계마다 처음부터 끝까지 역전파합니다.

오차는 각 단계(t)마다 오차를 측정하고 이전 단계로 전달되는데, 이것을 BPTT라고 합니다.

3에서 구한 오차를 이용하여 w_{xh} , w_{hh} , w_{hy} 및 바이어스(bias)를 업데이트합니다.



7.4 RNN 구조 — t 단계 계산 ③ 오차 & ④ 역전파(BPTT)

이진영 | AM202616703

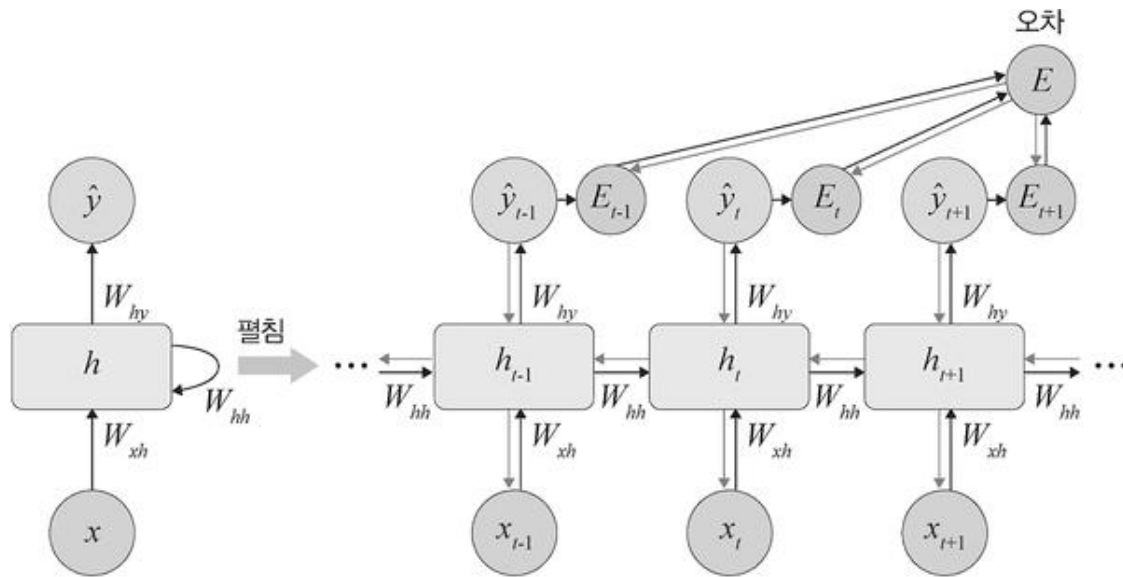
기울기 소멸 문제

BPTT는 오차가 멀리 전파될 때(왼쪽으로 전파)
계산량이 많아지고 전파되는 양이 점차 적어지
는 문제 → 기울기 소멸 문제(vanishing gradient
problem)가 발생합니다.

→ 해결책: LSTM 및 GRU를 많이 사용합니다.

✦ 생략된-BPTT (Truncated-BPTT)

계산량을 줄이기 위해 현재 단계에서 일정 시점까
지만(보통 5단계 이전까지만) 오류를 역전파하는
것을 생략된-BPTT라고 합니다.



7.4.1 RNN 셀 구현 — IMDB 데이터셋 & 라이브러리

이진영 | AM202616703

데이터 준비 ①

★ IMDB 데이터셋

IMDB 데이터셋은 영화 리뷰에 대한 데이터 5만 개로 구성되어 있습니다. 훈련 데이터 2만 5000개와 테스트 데이터 2만 5000개로 나누며, 각각 50%씩 긍정 리뷰와 부정 리뷰가 있습니다. 이 데이터는 이미 전처리가 되어 있어 각 리뷰가 숫자로 변환되어 있습니다.

코드 7-3 라이브러리 호출

```
import torch
import torchtext
import numpy as np
import torch.nn as nn
import torch.nn.functional as F
import time
```

코드 7-4 데이터 전처리

```
start = time.time()
TEXT = torchtext.legacy.data.Field(lower=True,
fix_length=200, batch_first=False) -----①
LABEL = torchtext.legacy.data.Field(sequential=False) -----②
```

Field 파라미터 설명

① TEXT 필드 파라미터

1. lower=True : 대문자를 모두 소문자로 변환 (기본값: False)
2. fix_length=200 : 데이터 길이를 200으로 고정, 200보다 짧으면 패딩 작업으로 200에 맞춤
3. batch_first=False : 입력 데이터 형태 = [시퀀스 길이, 배치 크기, 은닉층 뉴런 개수]
4. True로 설정 시 = [배치 크기, 시퀀스 길이, 은닉층 뉴런 개수]

② LABEL 필드 파라미터

1. sequential=False : 데이터에 순서가 있는지 나타냄
2. 레이블은 긍정/부정 값만 갖기 때문에 False로 설정

7.4.1 RNN 셀 구현 — 데이터셋 준비 & 전처리

이진영 | AM202616703

데이터 준비 ②

코드 7-5 데이터셋 준비

```
from torchtext.legacy import datasets
train_data, test_data = datasets.IMDB.splits(TEXT, LABEL) ----①
```

① datasets.IMDB를 사용하여 IMDB 데이터셋을 내려받습니다.

splits : 전체 데이터셋을 텍스트(TEXT)와 레이블(LABEL)로 분할합니다
이후 텍스트 데이터셋은 훈련 용도로, 레이블은 테스트 용도로 사용합니다.

코드 7-7 데이터셋 전처리 적용

```
import string

for example in train_data.examples:
    text = [x.lower() for x in vars(example)['text']] ----소문자로 변환
    text = [x.replace("<br />", "") for x in text] -- <br>을 공백으로 변환
    text = [''.join(c for c in s if c not in string.punctuation) for s
in text] # 구두점 제거
    text = [s for s in text if s] ----- 공백 제거
    vars(example)['text'] = text
```

코드 7-8 훈련과 검증 데이터셋 분리

```
import random
train_data, valid_data = train_data.split(
    random_state=random.seed(0), -----①
    split_ratio=0.8) -----②
```

① random_state : 데이터가 임의로 섞인 상태에서 분할
.시드(seed) 값을 사용하면 동일한 코드를 여러 번 수행
해도 동일한 결과 반환

② split_ratio=0.8 : 훈련과 검증 데이터셋을 8:2로 분리

코드 7-9 데이터셋 개수 확인 & 결과

```
print(f'Number of training examples:
{len(train_data)}')
print(f'Number of validation examples:
{len(valid_data)}')
print(f'Number of testing examples:
{len(test_data)}')

# 출력 결과
Number of training examples: 20000
Number of validation examples: 5000
Number of testing examples: 25000
```

훈련 2만 개 / 검증 5000개 / 테스트 2만 5000개로 분할

7.4.1 RNN 셀 구현 – 단어 집합 만들기 & 메모리 로딩

이진영 | AM202616703

데이터 준비 ③

코드 7-10 단어 집합 만들기

```
TEXT.build_vocab(train_data, max_size=10000, min_freq=10,
vectors=None) -----①
LABEL.build_vocab(train_data)

print(f"Unique tokens in TEXT vocabulary: {len(TEXT.vocab)}")
print(f"Unique tokens in LABEL vocabulary: {len(LABEL.vocab)}")

# 출력 결과
Unique tokens in TEXT vocabulary: 10002
Unique tokens in LABEL vocabulary: 3
```

① build_vocab() 파라미터

max_size=10000 : 단어 집합의 크기 (포함되는 어휘 수)
min_freq=10 : 훈련 데이터셋에서 특정 단어의 최소 등장 횟수. 최소 열 번 이상 등장한 것만 단어 집합에 추가
vectors=None : 임베딩 벡터 지정. Word2Vector, Glove 등 사전 학습된 임베딩 사용 가능

코드 7-11 단어 집합 확인

```
print(LABEL.vocab.stoi)
# 결과: {'<unk>': 0, 'pos': 1, 'neg': 2}
# pos(긍정), neg(부정) 외에 <unk>(사전에 없는 단어)가 포함됨
```

코드 7-12 데이터셋 메모리로 가져오기

```
BATCH_SIZE = 64
device = torch.device('cuda:0' if
torch.cuda.is_available() else 'cpu')

embedding_dim = 100 ----- 각 단어를 100차원으로 조정
hidden_size = 300 -----① 은닉층의 유닛 개수

train_iterator, valid_iterator, test_iterator = \
    torchtext.legacy.data.BucketIterator.splits(
        (train_data, valid_data, test_data),
        batch_size=BATCH_SIZE,
        device=device)
```

① hidden_size = 300 : 은닉층의 유닛 개수를 지정

② BucketIterator : 데이터로더(DataLoader)와 쓰임새가 같음. 배치 크기 단위로 값을 차례대로 꺼내어 메모리로 가져옴. 비슷한 길이의 데이터를 한 배치에 할당하여 패딩(padding)을 최소화

7.4.1 RNN 셀 구현 – 워드 임베딩 & RNN 셀 정의

이진영 | AM202616703

모델 정의

코드 7-13 워드 임베딩 및 RNN 셀 정의

```
class RNNCell_Encoder(nn.Module):
    def __init__(self, input_dim, hidden_size):
        super(RNNCell_Encoder, self).__init__()
        self.rnn = nn.RNNCell(input_dim, hidden_size) -----①

    def forward(self, inputs):
        bz = inputs.shape[1] # 배치를 가져옵니다.
        ht = torch.zeros((bz, hidden_size)).to(device) # 은닉층 뉴런
        크기를 0으로 초기화
        for word in inputs:
            ht = self.rnn(word, ht) -----②
        return ht

class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.em = nn.Embedding(len(TEXT.vocab.stoi), embedding_dim)
        -----③
        self.rnn = RNNCell_Encoder(embedding_dim, hidden_size)
        self.fc1 = nn.Linear(hidden_size, 256)
        self.fc2 = nn.Linear(256, 3)

    def forward(self, x):
        x = self.em(x)
        x = self.rnn(x)
```

파라미터 설명

① nn.RNNCell(input_dim, hidden_size)

- input_dim : 훈련 데이터셋의 특성(feature) 개수
- (배치, 입력 데이터 칼럼 개수(batch, input_size)) 형태
- hidden_size : 은닉층의 뉴런(유닛) 개수
- (배치, 은닉층의 뉴런 개수(batch, hidden_size)) 형태

② ht = self.rnn(word, ht)

- word : 현재의 입력 벡터 x_t
- (배치, 입력 데이터 칼럼 개수(batch, input_size)) 형태
- ht : 이전 상태 h_{t-1}
- (배치, 은닉층의 뉴런 개수(batch, hidden_size)) 형태

③ nn.Embedding(len(TEXT.vocab.stoi), embedding_dim)

- 첫 번째 파라미터 : 임베딩할 단어 수 (단어 집합의 크기)
- embedding_dim : 임베딩할 벡터의 차원

7.4.1 RNN 셀 구현 — 손실함수 & 학습 함수 정의

이진영 | AM202616703

모델 학습 ①

코드 7-14 옵티마이저와 손실 함수 정의

```
model      = Net()
model.to(device)

loss_fn    = nn.CrossEntropyLoss()  -----①
optimizer  = torch.optim.Adam(model.parameters(), lr=0.0001)
```

① nn.CrossEntropyLoss : 다중 분류에 사용됩니다.

nn.LogSoftmax와 nn.NLLLoss 연산의 조합으로 구성됩니다.

신경망에서 로그 확률 값을 얻으려면 마지막 계층에

LogSoftmax를 추가해야 합니다.

코드 7-15 모델 학습을 위한 함수 정의 (핵심 부분)

```
def training(epoch, model, trainloader, validloader):
    correct = 0; total = 0; running_loss = 0
    model.train()
    for b, batch in enumerate(trainloader):
        x, y = b.text, b.label
        x, y = x.to(device), y.to(device)
        y_pred = model(x)
        loss = loss_fn(y_pred, y) # CrossEntropyLoss로 오차 계산
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        with torch.no_grad():
            y_pred = torch.argmax(y_pred, dim=1)
            correct += (y_pred == y).sum().item()
            total += y.size(0)
            running_loss += loss.item()
    epoch_loss = running_loss / len(trainloader.dataset)
    epoch_acc = correct / total
    ...
    return epoch_loss, epoch_acc, epoch_valid_loss, epoch_valid_acc
```

7.4.1 RNN 셀 구현 — 손실함수 & 학습 함수 정의

이진영 | AM202616703

모델 학습 ①

코드 7-16 모델 학습

```
epochs      = 5
train_loss = []; train_acc = []
valid_loss = []; valid_acc = []

for epoch in range(epochs):
    epoch_loss, epoch_acc,
    epoch_valid_loss, epoch_valid_acc = \
        training(epoch, model,
                  train_iterator,
                  valid_iterator)
    train_loss.append(epoch_loss)
    train_acc.append(epoch_acc)
    valid_loss.append(epoch_valid_loss)
    valid_acc.append(epoch_valid_acc)

end = time.time()
print(end-start)
```

학습 결과

```
epoch: 0  loss: 0.011  accuracy: 0.498
valid_accuracy: 0.508
epoch: 1  loss: 0.011  accuracy: 0.502
valid_accuracy: 0.508
epoch: 2  loss: 0.011  accuracy: 0.514
valid_accuracy: 0.496
epoch: 3  loss: 0.011  accuracy: 0.515
valid_accuracy: 0.497
epoch: 4  loss: 0.011  accuracy: 0.524
valid_accuracy: 0.515
# 3079초 소요
```

에포크 5 → 정확도 낮지만 훈련/검증 오차 유사 → 과적합 없음

7.4.1 RNN 셀 구현 — 모델 예측 & 테스트 결과

이진영 | AM202616703

코드 7-17 모델 예측 함수 정의

```
def evaluate(epoch, model, testloader):
    test_correct = 0; test_total = 0; test_running_loss = 0
    model.eval()
    with torch.no_grad():
        for b in testloader:
            x, y = b.text, b.label
            x, y = x.to(device), y.to(device)
            y_pred = model(x)
            loss = loss_fn(y_pred, y)
            y_pred = torch.argmax(y_pred, dim=1)
            test_correct += (y_pred == y).sum().item()
            test_total += y.size(0)
            test_running_loss += loss.item()
    epoch_test_loss = test_running_loss / len(testloader.dataset)
    epoch_test_acc = test_correct / test_total
    print('epoch: ', epoch,
          'test_loss: ', round(epoch_test_loss, 3),
          'test accuracy:', round(epoch_test_acc, 3))
    return epoch_test_loss, epoch_test_acc
```

코드 7-18 모델 예측 결과 확인

```
# 출력 결과
epoch: 0 test_loss: 0.011 test_accuracy: 0.504
epoch: 1 test_loss: 0.011 test_accuracy: 0.504
... (이하 동일)
# 3315초 소요
```

모델 학습 ②

결과 분석 및 정리

훈련/검증 데이터셋 결과

- 에포크 5로 지정했기 때문에 정확도가 낮음
- 학습과 검증 데이터셋에 대한 오차가 유사하므로 과적합은 발생하지 않음

테스트 데이터셋 결과 : 약 50.4%

- 검증 데이터셋의 평가 결과와 크게 다르지 않음
- 더 높은 정확도를 원한다면 에포크 횟수를 늘릴 것

다음 단계

- 이 결과를 염두에 두고 RNN 계층을 적용했을 때의 결과와 비교 (7.4.2)

△ 에포크 수를 늘리거나 배치 크기와 같은 하이퍼파라미터를 조정하면 성능이 좋아질 수 있습니다. 다양한 모델을 적용한 후 결과가 가장 좋은 모델을 선택하는 것이 중요합니다.

7.4.2 RNN 계층 구현 — 데이터 준비 & 모델 정의

이진영 | AM202616703

RNN 계층

RNN 계층은 RNN 셀의 네트워크와 크게 다르지 않습니다. 동일한 IMDB 데이터셋을 사용합니다.

코드 7-20 데이터셋 내려받기 및 전처리

```
start = time.time()
TEXT = torchtext.legacy.data.Field(sequential=True,
batch_first=True, lower=True)
LABEL = torchtext.legacy.data.Field(sequential=False,
batch_first=True)

from torchtext.legacy import datasets
train_data, test_data = datasets.IMDB.splits(TEXT, LABEL)
train_data, valid_data = train_data.split(split_ratio=0.8)

TEXT.build_vocab(train_data, max_size=10000, min_freq=10,
vectors=None)
LABEL.build_vocab(train_data)

BATCH_SIZE = 100
device = torch.device('cuda:0' if torch.cuda.is_available()
else 'cpu')
vocab_size = len(TEXT.vocab)
n_classes = 2 # pos(긍정), neg(부정)
```

코드 7-21 데이터셋 분리

```
train_iterator, valid_iterator, test_iterator = \
    torchtext.legacy.data.BucketIterator.splits(
        (train_data, valid_data, test_data),
        batch_size=BATCH_SIZE, device=device)
```

7.4.2 RNN 계층 구현 — 데이터 준비 & 모델 정의

이진영 | AM202616703

코드 7-23 RNN 계층 네트워크

```
class BasicRNN(nn.Module):
    def __init__(self, n_layers, hidden_dim,
                  n_vocab, embed_dim,
                  n_classes, dropout_p=0.2):
        super(BasicRNN, self).__init__()
        self.n_layers = n_layers # RNN 계층 개수
        self.embed = nn.Embedding(n_vocab, embed_dim)
        self.hidden_dim = hidden_dim
        self.dropout = nn.Dropout(dropout_p)
        self.rnn = nn.RNN(embed_dim, self.hidden_dim,
                           num_layers=self.n_layers,
                           batch_first=True) -----①
        self.out = nn.Linear(self.hidden_dim, n_classes)

    def forward(self, x):
        x = self.embed(x) # 문자를 숫자/벡터로 변환
        h_0 = self._init_state(batch_size=x.size(0))
        x, _ = self.rnn(x, h_0)
        h_t = x[:, -1, :] # 마지막 은닉 상태의 값
        self.dropout(h_t)
        logit = torch.sigmoid(self.out(h_t))
        return logit

    def _init_state(self, batch_size=1):
        weight = next(self.parameters()).data
        return weight.new(
            self.n_layers, batch_size,
            self.hidden_dim).zero_()
```

BasicRNN: nn.RNN 사용 | batch_first=True → 배치 크기가 첫 번째 차원

7.4.2 RNN 계층 구현 — nn.RNN 파라미터 & 학습/평가 함수

이진영 | AM202616703

RNN 계층 학습

① nn.RNN 파라미터 설명

- embed_dim : 훈련 데이터셋의 특성(feature) 개수(칼럼 개수)
- self.hidden_dim : 은닉층의 뉴런 개수
- num_layers=self.n_layers : RNN 계층의 개수
- batch_first : 기본값 False → 입력 데이터 형태 = (시퀀스 길이, 배치 크기, 특성 개수)
- True 설정 시 → (배치 크기, 시퀀스 길이, 특성 개수) 형태로 변경

코드 7-24 손실 함수와 옵티마이저 설정

```
model = BasicRNN(n_layers=1, hidden_dim=256,
                 n_vocab=vocab_size,
                 embed_dim=128, n_classes=n_classes,
                 dropout_p=0.5)
model.to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
```

코드 7-25 모델 학습 함수 (핵심)

```
def train(model, optimizer, train_iter):
    model.train()
    for b, batch in enumerate(train_iter):
        x, y = batch.text.to(device),
        batch.label.to(device)
        y.data.sub_(1)  --① 레이블 값을 0과 1로 변환 (긍정
                        :2→1, 부정:1→0)
        optimizer.zero_grad()
        logit = model(x)
        loss = F.cross_entropy(logit, y)
        loss.backward()
        optimizer.step()
```

y.data.sub_(1): 레이블 2/1 → 1/0 변환 | logit.max(1)[1]: 예측 클래스 인덱스

7.4.2 RNN 계층 구현 — nn.RNN 파라미터 & 학습/평가 함수

이진영 | AM202616703

RNN 계층 학습

코드 7-26 모델 평가 함수

```
def evaluate(model, val_iter):
    model.eval()
    corrects, total, total_loss = 0, 0, 0
    for batch in val_iter:
        x, y = batch.text.to(device),
                batch.label.to(device)
        y.data.sub_(1)
        logit = model(x)
        loss = F.cross_entropy(
            logit, y, reduction="sum")
        total += y.size(0)
        total_loss += loss.item()
        corrects += (logit.max(1)[1]
                     .view(y.size()).data
                     == y.data).sum() -----①
    avg_loss = total_loss / len(val_iter.dataset)
    avg_accuracy = corrects / total
    return avg_loss, avg_accuracy
```

① 정확도 계산

- `logit.max(1)[1]` : 예측값 중 가장 큰 값의 인덱스 (예측 클래스)
- `.view(y.size())` : y의 크기에 맞게 형태 변환
- `.data == y.data` : 예측 결과와 실제 레이블 비교
- `.sum()` : 일치하는 개수를 `corrects`에 누적

7.4.2 RNN 계층 구현 – 학습 실행 & 최종 결과

이진영 | AM202616703

최종 결과

코드 7-27 모델 학습 및 평가

```
BATCH_SIZE = 100
LR          = 0.001
EPOCHS      = 5
for e in range(1, EPOCHS + 1):
    train(model, optimizer, train_iterator)
    val_loss, val_accuracy = evaluate(model, valid_iterator)
    print("[EPOCH: %d], Validation Loss: %5.2f | Validation
Accuracy: %5.2f"
          % (e, val_loss, val_accuracy))
```

학습 결과 (출력값)

```
Train Epoch: 1 [0/20000 (0%)]    Loss: 0.695734
Train Epoch: 1 [5000/20000 (25%)] Loss: 0.697002
Train Epoch: 1 [10000/20000 (50%)] Loss: 0.700943
Train Epoch: 1 [15000/20000 (75%)] Loss: 0.691708
[EPOCH: 1], Validation Loss: 0.69 | Validation Accuracy: 0.51
...
[EPOCH: 5], Validation Loss: 0.69 | Validation Accuracy: 0.50
```

코드 7-28 테스트 & 최종 결과

```
test_loss, test_acc = evaluate(model,
test_iterator)
print("Test Loss: %5.2f | Test Accuracy:
%5.2f" % (test_loss, test_acc))
```

출력 결과

```
Test Loss: 0.69 | Test Accuracy: 0.56
```


7.4.2 RNN 계층 구현 — 학습 실행 & 최종 결과

이진영 | AM202616703

최종 결과

결과 분석 & 결론

검증 데이터셋 결과 : 약 50% 정확도

- 여전히 높은 예측이라고 볼 수 없습니다
- 많은 에포크를 지정하고 하이퍼파라미터를 조정하면 성능이 좋아질 수 있습니다

테스트 데이터셋 결과 : 56% 정확도

- 여전히 모델 성능이 좋다고 할 수 없는 상황입니다
- 다른 모델로 변경하여 테스트를 진행해야 합니다

📌 RNN 셀 vs RNN 계층 비교

- RNN 셀 테스트 정확도 : 약 50.4%
- RNN 계층 테스트 정확도 : 56%

이렇게 여러 유형의 모델을 적용한 후 결과가 가장 좋은 모델을 선택합니다.
하이퍼파라미터를 튜닝해 나가는 과정이 필요하므로 데이터 분석은 꽤 많은 시간이 필요합니다.